

Sphery ExerCube SDK

Quickstart guide

The following document contains a minimal description of the requirements, capabilities and limitations of the Sphery ExerCube SDK. Please be aware that the Sphery ExerCube SDK is still under active development, so the contents are subject to change. All customers using the Sphery ExerCube SDK will be notified if a new version will become available.

Table of contents

Requirements.....	2
System.....	2
Unity3D Project.....	2
Installation.....	3
Getting Started.....	4
System Overview.....	4
ExerCube.....	4
SDK.....	4
Koboldgames Primitives Package.....	5
General Workflow.....	6
Scene Setup.....	6
Tracking Manager Initialization.....	7
Working with Tracker Data.....	8
Getting Tracker Data.....	8
Working with Poses.....	8
Creating Poses.....	8
Checking Poses.....	8

Requirements

To be able to use the Sphery ExerCube SDK, certain requirements need to be met. The following lists describe those requirements.

System

- [ExerCube](#)
To use the SDK, ExerCube-Hardware is not always required. Software can be developed without an ExerCube, but testing capabilities are very limited at the moment and every component of the SDK is tailored to be used within the ExerCube.
- **Computer**
To develop with the SDK, a computer is required. Currently, only Windows 10 is supported. This is because the ExerCube itself is based on a Windows 10 computer. The following software needs to be installed on the computer:
 - [Unity3D](#) – 2019.4.11f1
The SDK is based on Unity. The minimal required version is 2019.4.11f1 but it should work with newer versions.
 - [Steam](#)
The HTC Vive used by the ExerCube requires Steam to be installed, a Steam user account and connectivity with the internet at least once to download the Steam VR Runtime.
 - [Steam VR Runtime](#)
The HTC Vive requires the Steam VR Runtime to be installed.

Unity3D Project

A Unity Project that wants to use the Sphery ExerCube SDK needs to meet the following requirements:

- [SteamVR Unity Plugin](#) – v2.7.2 (sdk 1.14.15)
The SDK relies on the SteamVR Unity Plugin to receive tracking data from the HTC Vive. The minimum required version is already included in the SDK project.
- [Required packages](#)
Unity provides additional functionality using Packages. The Package Manager can be found under *Window > Package Manager*. Please ensure that the following packages are installed in the project.
 - **Koboldgames Primitives Package**
This package provides functionality for shared data. This package is not yet publicly available but already included in the SDK project's Packages folder.
 - **Koboldgames Non-Null Package**
This package provides functionality for detecting and alerting if data is required but null. This package is not yet publicly available but already included in the SDK project's Packages folder.

(sphery)

- **Cinemachine 2.3.4**
Cinemachine is used for controlling the Camera Rig.
- **TextMeshPro 2.1.3**
TextMeshPro is used for UI.
- **Universal RP 7.3.1**
Used for rendering with maximal hardware compatability. Unity's builtin renderer breaks under certain configurations.
- **UnityUI 1.0.0**
UnityUI is used for UI.
- **OpenVR XR Plugin 1.1.4**
OpenVR XR Plugin is required for Unity to communicate with the HTC Vive.

Installation

Due to the variety and complexity of dependencies, the Sphery ExerCube SDK is distributed as a Unity Project for now and should be installed in the following way:

- Ensure your **system meets the requirements** mentioned above.
- Create a new Unity project or just copy the whole project within the SDK package as a starter project.
 - For newly created projects, copy at least the contents from the **Assets** and **Packages** folder and ensure your project meets the **Unity3D project requirements** mentioned above. Make sure that SteamVR actions are generated (Window > Steam VR Input), otherwise the project cannot be built.

Getting Started

System Overview

ExerCube

The following components of the ExerCube are relevant to the SDK:

- **Walls as displays**
The three walls of the ExerCube are used as displays. They are projected upon by short distance projectors. The SDK provides functionality to configure the projection.
- **Tablet**
The tablet is an additional display available inside the SDK.
- **HTC Vive**
Tracking is done via the HTC Vive. The standard configuration of the ExerCube allows for tracking four trackers. One for each wrist and ankle.

SDK

The ExerCube SDK provides the following functionality:

- **Tracking + Player Calibration**
Trackers for the wrists and ankles can be tracked. There's a `TrackingManager` component that handles most of the work. Get a reference to it and implement your logic.
- **Poses**
Poses are data structures that describe the positions of the trackers. A pose matches if each tracker is within the corresponding bounds defined in the pose. Poses can be matched using the Tracking Manager. Some example poses can be found at *Assets/Exercube/Tracking/Poses/Examples*.
- **CameraRig + Displays**
The camera rig is composed of the 3 cameras projected on the walls. The cameras are automatically aligned on scene startup to match the configured display setup (Camera Rotator component). There is a separate camera used for the tablet. The Camera Render Target component is used on scene startup to match the display configuration.
- **UI Tracking**
The ExerCube SDK provides functionality to interact with UI elements (only Buttons for now) by pointing your arm at the element. The tablet's UI is excluded from this functionality. See the Tracking scene in *Assets/Exercube/Demo* for reference.
- **Demo Scenes**
There are two demo scenes included in the SDK. They can be found in the *Assets/Exercube/Demo* folder. *Demo.scene* should be your main reference point when working with the SDK. It shows how display calibration and tracking works. *Tracking.scene* shows how UIs need to be setup to use with tracking.

Koboldgames Primitives Package

This library introduces a way of decoupling data from the actual logic by providing different types of data as shared variables. The implementation is based on Unity's ScriptableObjects.

The most important concepts of this package are the following:

- **Shared Variables**
ScriptableObjects instances of different data types that can be created via Unity's Create Asset menu. The most common data types are included.
- **Flexible Variables**
Variable that can either be a reference to a Shared Variable or a local instance of the type. They are useful for testing without needing to change Shared Variables or for data that isn't necessarily shared.
- **Shared Collections**
Shared Collections are ScriptableObject instances of different collections of a certain type.
- **Shared Events + Event Listener Component**
Shared Events are ScriptableObject instances representing events that can be invoked from anywhere. Shared Events can be listened to by registering to the corresponding events in the object. There is also an Event Listener component provided that links a Shared Event to a Unity Event.

A more detailed description of the package can be found in the included README-File.

General Workflow

When working with the ExerCube SDK, you're free to create content on any of your Windows devices. To test your creation, build your Unity project for Windows and copy it on the computer included with the ExerCube.

Direct testing from your device (regarding tracking etc.) or a Unity installation on the ExerCube (regarding output to displays etc.) is not yet available, therefore it is vital that you output your debug information into a file or directly in your project's UI. Direct testing will become available in the future.

Scene Setup

Every Unity scene that you want to use with the ExerCube SDK has to have at least the following Prefabs referenced:

- **Player**
The player prefab contains all relevant components regarding tracking and a virtual camera that can be used by the CameraRig. The prefab can be found at *Assets/Exercube/Player/Player.prefab*.
- **CameraRig**
The camera rig contains all relevant components regarding the wall displays. The prefab can be found at *Assets/Exercube/Displays/Camera/CameraRig.prefab*.
- **TabletCamera**
The tablet camera prefab contains all relevant components regarding the tablet display. The prefab can be found at *Assets/Exercube/Displays/Camera/TabletCamera.prefab*.

The scene's origin point [0, 0, 0] corresponds to the cube's center point on the floor. All data (tracking and displays) is based on this configuration.

You can create a persistent scene and manage other scenes yourself if you want to. For beginners, it's easiest just to copy the *Demo.scene* in the *Assets/Exercube/Demo* folder and remove the objects you don't need.

Tracking Manager Initialization

The tracking manager needs to be initialized before it tracks correctly. There is an example script included that shows how this is done (*Assets/Exercube/Demo/Examples.cs*).

- **Search and Mapping** searches for all available trackers and maps them internally.

```
TrackingManager trackingManager;  
...  
trackingManager.StopActiveMapping();  
trackingManager.StartActiveMapping();  
trackingManager.SearchAndMappingActive
```

- **Calibrate Player** requires the player to stand in the middle of the ExerCube with their arms extended upwards to capture the player's height (*Assets/Exercube/Player/Calibration/PlayerSizeFactor.asset*). This is used for automatically adjusting the bounds of poses.

```
public PlayerCalibration playerCalibration;  
...  
playerCalibration.Calibrate();
```

- **Tracker Validation** ensures that tracker data is correct and usable by the tracking manager. Once started, it checks the data continuously.

```
public TrackingManager trackingManager;  
...  
trackingManager.ValidationRunning = true;
```

Working with Tracker Data

Getting Tracker Data

To get tracker data, you first need to get a reference to the tracking manager (can be found within the *Player.prefab*). Once the tracking manager is initialized, you can retrieve the tracker's data via index using the Tracker enumerator.

```
public TrackingManager trackingManager;  
...  
Vector3 position = trackingManager[Tracker.LeftWrist].Position;
```

Within the folder *Assets/Exercube/Tracking/Gestures* you can find some example uses of this way of working.

Working with Poses

Creating Poses

Poses are assets that can be created via Unity's asset creation menu under the menu point *ExerCube>Pose*. The most important data within pose assets are their bounds for each tracker (Hand Right, Hand Left, Ankle Right, Ankle Left). While a pose asset is selected, a preview of the bounds is shown within the scene view.

Currently, poses can not be changed at runtime. Creating temporary poses at runtime will be implemented in a future version.

Checking Poses

To check for poses, you first need to get a reference to the tracking manager (can be found within the *Player.prefab*). Once the tracking manager is initialized, you can check for pose matching using the *IsPoseMatching(Pose pose)* method.

```
public TrackingManager trackingManager;  
public Pose myPose;  
...  
bool isMatching = trackingManager.IsPoseMatching(myPose);
```